

PYTHON LABORATORY ASSESSMENT

EMPLOYEE ATTENDANCE & PAYROLL MANAGEMENT SYSTEM

Duration: 2 Hours | Maximum Marks: 40 | Target Topic: CRUD Operations, List & Dictionary Management, JSON Persistence & Automated Calculations

1. Context & Problem Scenario

A company requires a lightweight command-line utility to maintain employee attendance and salary records. The system should allow administrators to register employees, record attendance, calculate salary deductions, search employee records, modify details, and remove employees. The application must maintain active records in memory using a **list of dictionaries** and provide persistence using a JSON file named **employees.json**.

2. System Architecture & Data Model

The application maintains employee records using a list of dictionaries. Each employee must follow this structure:

```
employees = [
{"id": 1, "name": "Rahul Sharma", "department": "IT",
"salary": 50000.0, "days_present": 25, "status": "Eligible"},
{"id": 2, "name": "Priya Mehta", "department": "HR",
"salary": 45000.0, "days_present": 22, "status": "Regular"}
]
```

Employee Fields

Field	Type	Description
id	Integer	Auto-generated unique ID
name	String	Employee name
department	String	Employee department
salary	Float	Monthly salary
days_present	Integer	Number of days attended
status	String	Automatically calculated

Automated Attendance Status

The employee status must be calculated automatically. The user must not manually enter the status.

- Days Present $\geq 24 \rightarrow$ Eligible
- Days Present ≥ 20 and $< 24 \rightarrow$ Regular
- Days Present ≥ 15 and $< 20 \rightarrow$ Warning
- Days Present $< 15 \rightarrow$ Critical

3. Sample Employee Dataset

ID	Employee Name	Department	Salary	Days Present	Status
1	Rahul Sharma	IT	50000.00	25	Eligible
2	Priya Mehta	HR	45000.00	22	Regular
3	Arjun Kapoor	Finance	60000.00	17	Warning
4	Sneha Verma	IT	55000.00	27	Eligible
5	Karan Singh	Marketing	40000.00	12	Critical

4. Operational Modules & Functional Requirements

Implement an interactive, menu-driven CLI loop displaying the following choices:

[1] Add Employee | [2] View Employees | [3] Search Employee | [4] Update Employee | [5] Delete Employee | [6] Calculate Payroll | [7] Save to JSON | [8] Load from JSON | [9] Exit

Module A: Employee Registration

- Prompt for employee name, department, monthly salary, and days present.
- Automatically generate a unique incremental ID.
- Automatically calculate the attendance status.
- Construct the record dictionary and append it to the employee list.

Module B: Employee Directory & Search

- Display all employee records in a clean, aligned tabular format.
- Handle an empty employee list gracefully.
- Search by Employee ID using exact match OR by Employee Name / Department using case-insensitive substring matching.
- Display matching records or a clear "No employee records found" message.

Module C: Record Modification & Deletion

- Locate an employee by ID and allow updating name, department, salary, or days present.
- If days present changes, automatically recompute the attendance status.
- For deletion, display the employee details and request explicit confirmation (y/n) before removing the record.

Module D: Payroll Calculation

The company considers 26 working days per month. Calculate payroll using:

Daily Salary = Monthly Salary / 26

Absent Days = 26 – Days Present

Deduction = Daily Salary × Absent Days

Final Salary = Monthly Salary – Deduction

Display the calculated payroll information clearly for the selected employee.

5. JSON Persistence

Save Operation: Serialize the complete employee list into **employees.json** using Python's **json.dump()** with **indent=4** formatting.

Load Operation: Read **employees.json** using **json.load()** and restore the in-memory employee records. Handle missing or malformed files without crashing.

6. Data Integrity & Exception Handling Directives

- Name and Department must not be blank after trimming whitespace.
- Salary must be greater than 0.
- Days Present must satisfy $0 \leq \text{days_present} \leq 26$.
- Status must always be determined programmatically; manual status input is prohibited.
- Numerical inputs such as menu choice, ID, salary, and days present must be protected with try-except ValueError blocks.
- File operations must handle FileNotFoundError and json.JSONDecodeError.
- The CLI session must never terminate abnormally because of bad user input.

7. Recommended Modular Function Prototypes

```
def add_employee(employees, next_id):
def calculate_status(days_present):
def display_employees(employees):
def search_employees(employees, search_term):
def update_employee(employees, employee_id):
def delete_employee(employees, employee_id):
def calculate_payroll(employee):
def save_to_json(filepath, employees):
def load_from_json(filepath):
```

8. Evaluation Scheme (40 Marks)

Component	Marks
Menu-driven CLI & Program Flow	6
Employee Add & View Operations	6

Search Functionality	6
Update & Delete Operations	8
Automated Status & Payroll Calculation	6
JSON Persistence	5
Validation & Exception Handling	3
Total	40

End of Assessment